PDC — Plataforma de Dados Comercial

PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

Enfoque DevOps — Integração e Entrega Contínuas (CI/CD)

Versão 1.1

Campo	Descrição
Projeto	PDC — Plataforma de Dados Comercial
Gerente de Projeto	Gabriel da Cunha
Gerente/Responsável de GCS-DevOps	Gabriel da Cunha
Fábrica de Software / Squad	Engenharia de Dados — Gabriel da Cunha e André Felipe
Repositório principal	https://github.com/GChandrone/pdc-plataforma-dados-comercial.git

Histórico de Revisões

Data	Versão	Descrição	Autor
10/08/2026	1.0	Elaboração do documento com enfoque em práticas DevOps (CI/CD).	Luiz Camargo
23/09/2026	1.1	Elaboração do Plano de Gerenciamento de Configuração do projeto PDC.	Gabriel da Cunha

Sumário

Sumário 2
1. Introdução 4
1.1 Finalidade 4
1.2 Escopo 4
1.3 Definições, Acrônimos e Abreviações 4
1.4 Referências 5
1.5 Visão Geral 5
2. Cultura DevOps e Papéis na Gerência de Configuração 6
3. Ferramentas, Ambientes e Infraestrutura 6
3.1 Ferramentas do Pipeline 6
3.2 Ambientes e Infraestrutura 7
3.3 Estratégia de Branching e Versionamento 9
4. Identificação da Configuração 9
4.1 Itens de Configuração 9
4.2 Convenção de Nomenclatura 10
4.3 Estrutura do Repositório 10
5. Integração Contínua (CI) 11
5.1 Etapas do Pipeline de CI 11
5.2 Política de Pull Request e Revisão de Código 11
5.3 Critérios de Qualidade (Quality Gate) 11
6. Entrega e Implantação Contínua (CD) 12
6.1 Etapas do Pipeline de CD 12
6.2 Estratégias de Implantação 12
6.3 Rollback e Recuperação 12
7. Infraestrutura como Código (IaC) 12
7.1 Versionamento de Infraestrutura 13
7.2 Ambientes Efêmeros 13
8. Controle de Configuração e Mudança 13
8.1 Processo de Solicitação de Mudança 13
8.2 Comitê de Controle de Mudança (CCM) 13
9. Baselines e Releases 13
9.1 Geração Automática de Baseline 13
9.2 Notas de Versão (Release Notes) 14
10. Estimativa do Status de Configuração 14
10.1 Monitoramento e Observabilidade 14
10.2 Métricas DORA 14
10.3 Relatórios e Auditorias 14
11. Marcos 14

12. Treinamento e Recursos 15
13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros 15
14. Quadro-Resumo do Pipeline CI/CD 16

1. Introdução

Este Plano de Gerenciamento de Configuração (PGC) descreve as atividades de Gestão de Configuração de Software (GCS) que serão executadas ao longo do ciclo de vida do produto, com ênfase em práticas DevOps — em particular Integração Contínua (CI), Entrega/Implantação Contínua (CD), Infraestrutura como Código (IaC) e automação de pipelines. As atividades envolvem identificar os itens de configuração, manter sua integridade e controlar sistematicamente as mudanças, com o mínimo de burocracia possível e o máximo de automação.

1.1 Finalidade

Estabelecer um padrão a ser seguido pela equipe, garantindo rastreabilidade, versionamento e controle de mudanças do produto, apoiado por pipelines automatizados de build, teste, integração e implantação.

1.2 Escopo

Este PGC se aplica ao projeto PDC — Plataforma de Dados Comercial, desenvolvido e mantido pelos engenheiros de dados Gabriel da Cunha (responsável) e André Felipe.

O resultado esperado é uma plataforma de dados analíticos do domínio Comercial que disponibiliza, em um painel Databricks AI/BI Dashboard atualizado diariamente, indicadores de pedidos de venda e carteira por unidade de negócio (Matriz, Acessórios, Nordeste, Centro-Oeste e TopMax), cliente, vendedor, produto e período. Os dados são 100% fictícios e simulam o ERP Protheus.

Estão no escopo os ambientes de Desenvolvimento e Produção, os itens de configuração da seção 4.1 e os processos de integração, entrega, controle de mudanças e auditoria descritos neste plano. Estão fora do escopo dados reais, sistemas reais e outros domínios de negócio.

1.3 Definições, Acrônimos e Abreviações

Termo	Significado
GCS	Gestão de Configuração de Software
CI	Integração Contínua (Continuous Integration)
CD	Entrega/Implantação Contínua (Continuous Delivery / Deployment)
IaC	Infraestrutura como Código (Infrastructure as Code)
CCM	Comitê de Controle de Mudanças
Baseline	Conjunto de itens de configuração que atingiram um estado comprovado de estabilidade
Pipeline	Sequência automatizada de etapas de build, teste, análise e implantação
Trunk-based Development	Estratégia de branching em que os desenvolvedores integram na branch principal com alta frequência
SemVer	Versionamento Semântico (MAJOR.MINOR.PATCH), ex.: 1.0.0
DORA Metrics	Métricas de desempenho DevOps (Lead Time, Deployment Frequency, MTTR, Change Failure Rate)
Medalhão	Organização em camadas: Bronze (espelho da origem), Silver (origem tratada) e Gold (modelo de negócio)

Termo	Significado
Free Edition	Versão gratuita do Databricks, somente com computação serverless e um workspace por conta
ERP simulado	Banco PostgreSQL local que simula o ERP Protheus, com dados fictícios, usado como fonte
Extrator	Script Python que lê o PostgreSQL de forma incremental e envia arquivos Parquet ao Databricks
Airflow / DAG	Apache Airflow, orquestrador que executa fluxos de tarefas (DAGs) agendados
Landing	Volume do Unity Catalog que recebe os arquivos Parquet enviados pelo extrator
Databricks Jobs	Orquestrador nativo do Databricks (agendamento e encadeamento de tarefas)
DAB	Databricks Asset Bundles — definição como código de jobs, schemas, notebooks e dashboards
Unity Catalog	Catálogo e governança de dados do Databricks (catálogo.schema.tabela)
Watermark	Último valor de S_T_A_M_P_ processado, usado para buscar apenas registros novos ou alterados
IngestionControl	Tabela de controle da extração (tabelas ativas, tipo de carga e watermark)
ADR	Architecture Decision Record — registro curto de uma decisão de arquitetura e sua justificativa

1.4 Referências

- Documento base: Processos de Desenvolvimento de Software e a Integração com GCS.
- Template do Plano de Gerenciamento de Configuração (RUP 7.0, IBM) — adaptado.
- Cronograma de marcos do projeto (seção 11 deste documento).
- Documentação oficial: GitHub Actions, Databricks Asset Bundles, Databricks Jobs, Databricks Free Edition, Unity Catalog, Apache Airflow, Docker Compose e PostgreSQL.
- Semantic Versioning 2.0.0 (semver.org) e Conventional Commits 1.0.0 (conventionalcommits.org).
- FORSGREN, N.; HUMBLE, J.; KIM, G. Accelerate. IT Revolution, 2018 (métricas DORA).
- BRASIL. Lei nº 13.709/2018 — Lei Geral de Proteção de Dados Pessoais (LGPD).

1.5 Visão Geral

Diferente de processos tradicionais em que a GCS é predominantemente manual e concentrada em marcos formais, neste plano a GCS é tratada como parte constante do pipeline de entrega: cada commit integrado, testado e implantado automaticamente gera evidências de rastreabilidade (logs de pipeline, tags de versão, artefatos publicados), reduzindo o esforço manual e o tempo entre a mudança e sua entrega em produção.

Visão da solução: um banco PostgreSQL local simula o ERP, com tabelas por empresa (ex.: SC5010, SC5080) e um simulador que gera a movimentação diária (novos pedidos, alterações e exclusões lógicas). Um extrator em Python, orientado pela tabela de controle IngestionControl, lê esse banco — de forma incremental pelo S_T_A_M_P_ nos pedidos (SC5 e SC6) e integral nos cadastros (SA1, SA3 e SB1) — e envia arquivos Parquet para um volume de landing no Databricks Free Edition. No Databricks, a Bronze incorpora os arquivos novos, a Silver trata e padroniza os dados e a Gold (gold.comercial) entrega o modelo dimensional consumido pelo dashboard. O Apache Airflow orquestra o fluxo diário — executa o extrator e, em seguida, aciona o job de transformação no Databricks —, e o GitHub Actions implanta a mesma versão em Desenvolvimento e Produção. PostgreSQL e Airflow rodam localmente em containers Docker.

Por ser um produto de dados, “build” é a validação dos notebooks, jobs e demais definições do bundle; “deploy” é a publicação desses artefatos no ambiente de destino; e “testes” incluem validações de qualidade dos dados.

2. Cultura DevOps e Papéis na Gerência de Configuração

A GCS em um contexto **DevOps é responsabilidade compartilhada entre desenvolvimento e operações ("Você constrói, você executa" — you build it, you run it), apoiada por automação.** Os papéis abaixo descrevem responsabilidades típicas, que podem ser exercidas por uma mesma pessoa em squads pequenas.

Papel	Responsabilidades
Gerente de Projeto / Product Owner	Prioriza o backlog (GitHub Projects), valida as regras de negócio dos indicadores e autoriza mudanças de maior impacto. Responsável: Gabriel da Cunha.
Engenheiro DevOps / Release Engineer	Mantém os workflows do GitHub Actions, a proteção da main, o versionamento, os Environments, o Asset Bundle e o ambiente Docker Compose. Responsável: Gabriel da Cunha.
Desenvolvedor(a)	Desenvolve o extrator, as DAGs do Airflow, notebooks, jobs e dashboard, escreve testes e integra via Pull Request. Responsáveis: Gabriel da Cunha e André Felipe.
SRE / Operações (Site Reliability Engineering)	Monitora as execuções em Produção, acompanha o horário de atualização dos dados, trata alertas e executa rollback. Responsável: André Felipe.
QA / Automação de Testes	Mantém os testes automatizados e os gates de qualidade de dados executados no CI e em cada carga. Responsável: André Felipe.
Segurança (DevSecOps)	Garante que tokens fiquem apenas em cofres de segredos, mantém a varredura de segredos e dependências e assegura o uso exclusivo de dados fictícios (LGPD). Responsável: Gabriel da Cunha.
Comitê de Controle de Mudanças (CCM)	Avalia apenas mudanças de alto impacto (seção 8.2). Composição: Gabriel da Cunha e André Felipe.

Por ser uma equipe pequena, o autor pode aprovar e mesclar o próprio PR; a qualidade é garantida pelo CI obrigatório. A revisão pelo outro engenheiro é recomendada e obrigatória nas mudanças do CCM.

3. Ferramentas, Ambientes e Infraestrutura

3.1 Ferramentas do Pipeline

Ferramentas adotadas no projeto (todas gratuitas):

Categoria	Ferramenta(s) adotada(s)	Finalidade
Controle de versão	Git + GitHub (repositório público, viável por usar apenas dados fictícios)	Versionamento de código, jobs, dashboard e configurações
Orquestração de CI/CD	GitHub Actions + GitHub Environments (dev e prd)	Validação, testes e implantação automatizados

Categoria	Ferramenta(s) adotada(s)	Finalidade
Empacotamento	Databricks Asset Bundles	Implantação versionada de notebooks, jobs, schemas e dashboard
Registro de artefatos	GitHub Releases (tag vX.Y.Z)	Versão imutável promovida de Dev para Produção
Infraestrutura como Código	Databricks Asset Bundles + script SQL idempotente de setup	Criação de catálogos, schemas, volumes, jobs e dashboard
Orquestração de containers	Docker + Docker Compose	Execução local do PostgreSQL do ERP simulado e do Airflow
Gestão de segredos	GitHub Secrets (tokens de deploy) + arquivo .env local do Docker Compose, fora do versionamento	Credenciais fora do código (secrets do Environment prd liberados só após a aprovação)
Monitoramento e Observabilidade	Interface e alertas do Airflow, notificações do Databricks Jobs e status na IngestionControl	Detecção de falhas e acompanhamento das cargas
Segurança / SCA / SAST	Dependabot, Bandit e gitleaks	Vulnerabilidades em dependências, código e segredos
Rastreamento de tarefas	GitHub Issues + GitHub Projects	Vinculação de commits/PRs a itens de trabalho
Plataforma de dados	Databricks Free Edition (serverless, Delta Lake, Unity Catalog)	Ingestão e camadas Bronze, Silver e Gold
Orquestração de dados	Apache Airflow (provider Databricks) + Databricks Jobs	DAGs diárias dag_erp_simulador (movimentação do ERP) e dag_comercial_diario (extração e acionamento do job_comercial_diario)
Fonte fictícia	PostgreSQL local + scripts de carga inicial e simulação	Simulação do Protheus multiempresa e da sua movimentação diária
Extração	Extrator Python (psycopg, pyarrow e Databricks SDK), executado como tarefa da DAG	Leitura incremental do PostgreSQL e envio de Parquet ao volume de landing
Ingestão	COPY INTO executado pelo Databricks Jobs	Incorporação idempotente dos arquivos novos da landing na Bronze
Visualização	Databricks AI/BI Dashboard	Consumo da camada Gold
Qualidade de código	ruff, pytest e pre-commit (com nbstripout)	Lint, testes e remoção de outputs dos notebooks antes do commit

3.2 Ambientes e Infraestrutura

Os ambientes devem ser, sempre que possível, provisionados via Infraestrutura como Código, garantindo paridade entre eles e permitindo recriação rápida e rastreável.

O projeto usa dois ambientes, Desenvolvimento e Produção, no mesmo workspace Databricks Free Edition, separados logicamente por catálogos e pelos targets dev e prd do Asset Bundle. A estrutura dos dois é criada pelo mesmo bundle e pelo mesmo script de setup.

Ambiente	Finalidade	Gatilho de promoção
Desenvolvimento	Integração e testes com a base fictícia; recebe todo merge na main.	Merge na main com CI verde
Homologação / Staging	Não adotado: os testes de aceitação e de qualidade de dados rodam em Desenvolvimento.	N/A
Produção	Ambiente consumido pelo dashboard; carga diária acionada pela DAG do Airflow.	Tag vX.Y.Z + aprovação manual no Environment prd

Detalhamento por ambiente:

- **Workspace:** um único workspace Free Edition, com acesso para os dois engenheiros.
- **Catálogos:** camada = catálogo, com prefixo dev_ em Desenvolvimento (dev_bronze, dev_silver, dev_gold) e sem prefixo em Produção (bronze, silver, gold), além do catálogo landing, compartilhado, que recebe os arquivos do extrator.
- **Targets do bundle:** dev em modo development (jobs com prefixo do usuário e agendamentos pausados) e prd em modo production (nomes definitivos; o job é acionado pelo Airflow). Os nomes dos catálogos são passados por variável do target, e nenhum notebook ou SQL cita um catálogo de forma fixa.
- **Fonte e extração:** um único PostgreSQL local (Docker Compose), compartilhado pelos dois ambientes, como um ERP real. A IngestionControl e os watermarks ficam no próprio PostgreSQL, sob controle do extrator, que grava em landing.protheus.arquivos, com um manifesto de contagem de registros por arquivo. A Bronze de cada ambiente lê a mesma landing com COPY INTO, que controla por ambiente os arquivos já carregados; nenhum pipeline grava na landing.
- **Carga inicial do ERP:** script executado uma única vez que cria os cadastros por empresa (clientes, vendedores e produtos) e um histórico de 12 meses de pedidos, com CPF/CNPJ de dígito verificador inválido.
- **Simulador do ERP:** script Python executado pela DAG dag_erp_simulador, antes da extração, que representa o uso diário do Protheus em cada empresa e filial: (1) inclui novos pedidos (SC5) com 1 a 5 itens (SC6), usando clientes, vendedores e produtos existentes; (2) altera parte dos pedidos em aberto (quantidade, preço ou faturamento); (3) exclui logicamente alguns pedidos (D_E_L_E_T_ = '*', como no Protheus); e (4) ocasionalmente inclui ou altera cadastros. Os volumes e percentuais ficam em conf/simulador.json e a semente aleatória é configurável, para execuções reprodutíveis nos testes.
- **Regras do simulador:** nunca exclui fisicamente nem reutiliza R_E_C_N_O_; itens só existem para cabeçalhos válidos; C6_VALOR = quantidade × preço; o S_T_A_M_P_ é atualizado pelo trigger do PostgreSQL em toda inclusão ou alteração, e não pelo script.
- **Orquestração:** a DAG dag_comercial_diario, no Airflow local, roda uma vez ao dia: a primeira tarefa executa o extrator e, só após o sucesso, a segunda aciona o job_comercial_diario de Produção (DatabricksRunNowOperator) e aguarda sua conclusão; o job executa Bronze, Silver, Gold e os gates de qualidade. Em Desenvolvimento, o job é executado pelo CD após cada deploy, fora do horário de Produção, respeitando o limite de 5 tarefas simultâneas da Free Edition, compartilhado pelos dois ambientes.
- **Containers locais:** um docker-compose.yml sobe o PostgreSQL do ERP simulado e o Airflow (com seu banco de metadados), a partir de uma imagem do Airflow que já inclui o extrator e o provider do Databricks.

- **Alterações em Produção:** nenhum objeto é criado ou alterado manualmente; tudo passa pelo CD, a partir de uma tag.
- **Dashboard:** um por ambiente, publicado pelo bundle e apontando para o catálogo gold do seu ambiente.
- **Credenciais:** token de deploy apenas nos GitHub Secrets dos Environments dev e prd; token do extrator e senha do PostgreSQL no arquivo .env do Docker Compose, fora do versionamento (a conexão do Airflow com o Databricks também é definida por ele); nenhum segredo neste documento ou no repositório.

3.3 Estratégia de Branching e Versionamento

Adota-se o GitHub Flow, com branches de vida curta:

- **main** — sempre implantável; protegida por PR obrigatório e CI verde, sem push direto.
- **feature/<issue>-<descrição>** — uma branch curta por item de trabalho, excluída após o merge.
- **release/x.y** — não adotada; a promoção para Produção é feita por tag.
- **hotfix/<descrição>** — correção emergencial a partir da tag em Produção, liberada como versão PATCH e integrada à main no mesmo PR.

Versionamento: SemVer aplicado ao modelo Gold — **MAJOR** para mudança incompatível (ex.: remover coluna ou mudar o grão), **MINOR** para nova tabela ou coluna e **PATCH** para correção. Commits no padrão Conventional Commits (feat, fix, docs, refactor, test, ci, chore).

4. Identificação da Configuração

4.1 Itens de Configuração

Além do código-fonte, em um contexto DevOps os seguintes artefatos também são itens de configuração formais, versionados no mesmo repositório (ou repositórios compartilhados):

- Código-fonte: extrator Python, SQL de ingestão da Bronze, notebooks Silver (um por tabela) e Gold, em .ipynb sem outputs, e testes automatizados
- Definições de pipeline como código: .github/workflows e databricks.yml (bundle)
- Definição dos jobs do Databricks (resources/ do bundle) e DAGs do Airflow
- Script SQL idempotente de setup (catálogos, schemas e volume de landing)
- Configurações do extrator e do simulador e conteúdo da IngestionControl (conf/ingestao.json e conf/simulador.json), com modelo .env.example — sem segredos
- Ambiente local: docker-compose.yml (PostgreSQL e Airflow), Dockerfile da imagem do Airflow, DDL das tabelas (com trigger do S_T_A_M_P_), carga inicial e simulador de movimentação
- Documentação técnica: README, ADRs e dicionário de dados
- Definição do dashboard (.lvdash.json)
- Arquivos de governança do repositório: .gitignore, .env.example (modelo sem valores reais), .pre-commit-config.yaml, CODEOWNERS e templates de PR e de issue

Não são itens de configuração: os dados das tabelas (reprodutíveis a partir dos artefatos), o watermark (estado de execução), os outputs de notebooks e os tokens de acesso. O .gitignore impede que esses e outros arquivos locais sejam versionados: o .env (senhas e tokens), a pasta .databricks/ (estado local do bundle), arquivos .parquet e saídas locais do extrator (landing_local/), caches do Python (__pycache__, .pytest_cache, .ruff_cache), ambientes virtuais (.venv) e arquivos de IDE e do sistema operacional.

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Branch de feature	feature/<issue>-<descrição>	feature/12-fato-pedido-venda
Commit	<tipo>(<escopo>): <descrição>	feat(gold): cria fato_pedido_venda
Tag de release	vX.Y.Z	v1.0.0
DAG	dag_<domínio>_<frequência>	dag_comercial_diario, dag_erp_simulador
Job	job_<domínio>_<frequência>	job_comercial_diario
Notebook	<camada>_<tabela ou entidade>	silver_sc6, gold_fato_pedido_venda
Catálogos	[dev_]<camada>; landing compartilhada	dev_silver (Desenvolvimento), silver (Produção)
Arquivos na landing	<tabela>/empresa=<emp>/<tabela>_<data-hora>.parquet	sc5/empresa=08/sc5_20261013T0600.parquet
Tabelas	<catálogo>.<sistema | domínio>.<tabela>	silver.protheus.sc6, gold.comercial.fato_pedido_venda
Colunas (Silver/Gold)	Prefixo semântico: cod_, dat_, qtd_, vlr_, des_…	cod_pedido, vlr_total
IngestionControl	tabela + empresa	SC5 / 08
Tabela de origem	<ALIAS><EMPRESA>0	SC5080 (pedidos da empresa 08)

4.3 Estrutura do Repositório

Monorepositório único, para que uma tag represente a versão completa do projeto (pdc-plataforma-dados-comercial/):

Caminho	Conteúdo
.github/	workflows/ (ci.yml, cd.yml), ISSUE_TEMPLATE/, pull_request_template.md, dependabot.yml e CODEOWNERS
databricks.yml	Asset Bundle, com os targets dev e prd
resources/	Jobs (job_comercial_diario e job_setup) e dashboard
src/	ingestao/, silver/, gold/, qualidade/, dashboards/ e pdc_lib/
setup/	setup.sql e executar_setup (catálogos, schemas e volume de landing)
docker-compose.yml	Ambiente local: PostgreSQL do ERP simulado e Airflow
airflow/	dags/ e Dockerfile da imagem do Airflow
erp_simulado/	ddl/, carga inicial e simulador de movimentação
extrator/	Extrator Python (pyproject.toml)
conf/	ingestao.json e simulador.json
tests/	unit/ e integration/
docs/	adr/ e dicionario-dados.md

Caminho	Conteúdo
.gitignore	Exclui do Git: .env, .databricks/, *.parquet, saídas locais, caches, ambientes virtuais e IDE
.env.example	Modelo das variáveis locais, sem valores reais
Raiz	README.md, CHANGELOG.md, pyproject.toml, requirements-dev.txt e .pre-commit-config.yaml

5. Integração Contínua (CI)

Todo Pull Request para a branch principal deve disparar automaticamente um pipeline de CI. Nenhuma mudança é integrada sem que o pipeline seja concluído com sucesso.

Localmente, hooks de pre-commit (ruff, gitleaks e nbstripout) antecipam parte das verificações e removem os outputs dos notebooks antes do commit.

5.1 Etapas do Pipeline de CI

1. Checkout do código e instalação das dependências Python (com cache).
2. Validação do bundle (databricks bundle validate) para os targets dev e prd.
3. Análise estática com ruff, teste de importação das DAGs e build da imagem do Airflow (docker compose build).
4. Testes unitários (pytest) do extrator (lógica de watermark), do simulador (regras de integridade) e das funções de transformação.
5. Testes de integração: o PostgreSQL sobe como service container com o DDL do ERP simulado; o extrator é executado duas vezes para comprovar que a carga incremental não perde nem duplica registros, e as transformações são validadas com PySpark local.
6. Análise de segurança: Bandit (SAST), Dependabot (SCA) e gitleaks (segredos). O bundle é implantado a partir do mesmo commit validado.
7. Verificação de isolamento: o CI falha se algum notebook ou SQL citar um catálogo de forma fixa, em vez de recebê-lo por parâmetro, ou se algum código do pipeline gravar na landing.

5.2 Política de Pull Request e Revisão de Código

- Toda mudança entra por PR. O autor pode aprovar e mesclar o próprio PR após o CI verde; a revisão pelo outro engenheiro é recomendada e obrigatória nas mudanças do CCM.
- PRs devem ser pequenos, de vida curta, referenciar a issue (Closes #n) e seguir o checklist do template de PR: testes, impacto na Gold e ausência de dados reais e segredos.
- Merge somente com o CI verde (quality gate), via squash merge.

5.3 Critérios de Qualidade (Quality Gate)

Critérios para o merge e para a promoção a Produção:

- Zero erros de lint, todos os testes passando e nenhum catálogo citado de forma fixa no código.
- Zero segredos detectados e zero vulnerabilidades críticas ou altas nas dependências.
- Gates de qualidade de dados (em Desenvolvimento e a cada carga): reconciliação da contagem do manifesto com a Bronze, zero nulos e duplicidades na chave (empresa + R_E_C_N_O_) e integridade da fato com as dimensões.

- Tabelas e colunas principais da Gold com COMMENT preenchido (descoberta no Unity Catalog).

6. Entrega e Implantação Contínua (CD)

Após o merge, o pipeline de CD implanta a mesma versão do bundle no target de Desenvolvimento e, após aprovação, no de Produção. O ambiente local (extrator, DAGs e imagem do Airflow) é atualizado a partir da mesma tag, com git checkout da versão e docker compose up --build (item do checklist da release).

6.1 Etapas do Pipeline de CD

1. Deploy automático em Desenvolvimento a cada merge na main (setup.sql + databricks bundle deploy -t dev).
2. Execução do job em Desenvolvimento (databricks bundle run) com os gates de qualidade de dados.
3. Gate para Produção: criação da tag vX.Y.Z e aprovação manual no GitHub Environment prd.
4. Deploy da mesma tag em Produção (databricks bundle deploy -t prd).
5. Smoke test: execução do job e verificação de que a Bronze incorporou os arquivos da última extração.
6. Publicação do GitHub Release com notas de versão geradas automaticamente (baseline).

6.2 Estratégias de Implantação

Estratégia	Descrição
Rolling update	Não aplicável: plataforma serverless, sem instâncias a substituir.
Blue-Green	Não adotado; em mudanças MAJOR, a nova tabela Gold pode ser criada em paralelo antes da troca.
Canary Release	Não aplicável: não há tráfego de usuários a dividir.
Feature Flags	Adotado de forma simples: a coluna Ativo da IngestionControl liga ou desliga a extração de cada tabela sem novo deploy.

Estratégia padrão: implantação direta, fora do horário da carga diária.

6.3 Rollback e Recuperação

Toda implantação deve ser reversível:

- **Código:** reimplantação da tag anterior pelo workflow de CD e, no ambiente local, docker compose up --build a partir da tag anterior.
- **Dados:** RESTORE das tabelas Delta para a versão anterior (Time Travel) e, se necessário, retrocesso do UltimoWatermark na IngestionControl para uma nova extração.
- **Contenção:** enquanto a correção não é implantada, a extração da tabela afetada pode ser desligada pela coluna Ativo da IngestionControl.

O procedimento de rollback de dados é documentado no README e registrado na issue do incidente.

7. Infraestrutura como Código (IaC)

Como toda a solução roda no Databricks Free Edition, a infraestrutura como código é o próprio Asset Bundle, que declara em YAML os jobs, schemas, volumes e o dashboard, complementado por um script SQL idempotente (CREATE ... IF NOT EXISTS) para catálogos e volume de landing, já que a Free Edition não permite criar catálogos pelo bundle. No ambiente local, o docker-compose.yml, o Dockerfile e o DDL descrevem o PostgreSQL e o Airflow. Todos seguem o mesmo fluxo de PR da aplicação.

7.1 Versionamento de Infraestrutura

- Toda alteração de infraestrutura passa por PR: databricks bundle validate no CI; deploy após o merge (Desenvolvimento) e após aprovação (Produção). Ajustes feitos pela interface são proibidos em Produção; qualquer divergência é corrigida com um novo deploy.
- O estado de implantação do bundle é mantido pelo próprio Databricks, separado por target, nunca em máquinas locais.

7.2 Ambientes Efêmeros

Não adotados, para manter o projeto simples. Em Desenvolvimento, o isolamento entre os dois engenheiros é feito pelo modo development do Asset Bundle, que prefixa os recursos com o nome do usuário.

8. Controle de Configuração e Mudança

8.1 Processo de Solicitação de Mudança

Fluxo padrão: issue (templates Funcionalidade, Correção ou Mudança de dados) → branch feature → Pull Request → CI → merge → CD em Desenvolvimento → tag → Produção. O histórico desse fluxo no GitHub é o registro formal e rastreável da mudança, da solicitação à implantação.

8.2 Comitê de Controle de Mudança (CCM)

O CCM é acionado apenas em mudanças de alto impacto ou risco elevado, como alterações estruturais de infraestrutura, mudanças de esquema de banco de dados em produção ou exceções ao fluxo padrão (ex.: hotfix emergencial). Em situações rotineiras, o fluxo automatizado de PR e CI substitui o comitê formal.

Composição: Gabriel da Cunha e André Felipe. O CCM é acionado em:

- mudança incompatível no modelo Gold (versão MAJOR);
- alteração de regra de negócio de indicador;
- remoção de objetos em Produção, reprocessamento completo ou RESTORE de tabelas;
- hotfix fora do fluxo padrão.

A decisão é registrada na issue com a label ccm e, quando envolver arquitetura, em um ADR.

9. Baselines e Releases

9.1 Geração Automática de Baseline

Diferente de modelos tradicionais, em que a baseline é criada manualmente em marcos do projeto, neste plano cada implantação bem-sucedida em produção gera automaticamente uma baseline: a tag de versão

(SemVer) e seu GitHub Release, que identificam o commit implantado (extrator, notebooks, jobs, dashboard e scripts de setup).

9.2 Notas de Versão (Release Notes)

As notas de versão são geradas automaticamente pelo GitHub Release, a partir dos PRs e commits desde a última tag, e mantidas também no CHANGELOG.md.

10. Estimativa do Status de Configuração

10.1 Monitoramento e Observabilidade

O ambiente de Produção é monitorado continuamente:

- **Falhas:** interface do Airflow (status de cada tarefa da DAG, com alerta de falha) e histórico do Databricks Jobs.
- **Extração:** log da tarefa do extrator no Airflow e colunas StatusUltimaCarga e DataUltimaExecucao da IngestionControl, por tabela.
- **Atualização:** o dashboard exibe a data e a hora da última carga da Gold, o que evidencia atrasos quando o ambiente local não rodou.
- **Objetivos de serviço:** Gold de Produção atualizada até as 7h e taxa de sucesso das cargas diárias de pelo menos 95% no mês.

10.2 Métricas DORA

Métrica	Descrição
Deployment Frequency	Frequência de deploys em Produção. Meta: 1 por semana.
Lead Time for Changes	Tempo entre o commit e o deploy em Produção. Meta: menos de 2 dias.
Change Failure Rate	Percentual de deploys que exigem rollback ou hotfix. Meta: abaixo de 15%.
Mean Time to Restore (MTTR)	Tempo médio para restaurar a carga após uma falha. Meta: menos de 4 horas.

As métricas são calculadas a partir do histórico de workflows e releases do GitHub.

10.3 Relatórios e Auditorias

O histórico de pipelines (builds, testes, deploys) e os logs do sistema de versionamento constituem, por si, o principal insumo de auditoria de GCS, dispensando relatórios manuais na maior parte dos casos.

Complementarmente, a cada release MAJOR e ao final do projeto é feita uma auditoria de configuração: verifica-se que a versão em Produção corresponde à última tag e que os gates de qualidade de dados continuam atendidos. O resultado é registrado em uma issue.

11. Marcos

Marcos do projeto:

Marco	Entrega	Data prevista
M1 — Repositório e CI	Repositório, main protegida, CI e teste de deploy do bundle nos targets dev e prd	29/09/2026
M2 — Ambiente local	Docker Compose com PostgreSQL multiempresa, simulador de movimentação, IngestionControl e Airflow	13/10/2026
M3 — Extração e ingestão	Extrator com cargas FULL e INCREMENTAL, landing, Bronze e DAG no Airflow	27/10/2026
M4 — Transformação	Silver e Gold do Comercial, acionadas pela DAG	10/11/2026
M5 — CD e release	Deploy automatizado e v1.0.0 em Produção	17/11/2026
M6 — Entrega final	Dashboard, métricas DORA, auditoria de configuração e documentação	01/12/2026

12. Treinamento e Recursos

Pessoal: Gabriel da Cunha e André Felipe, engenheiros de dados, com os papéis da seção 2.

Ferramentas: uma conta Databricks Free Edition, GitHub (plano gratuito), Docker Desktop, Python e VS Code — custo zero.

Treinamento (documentação oficial):

- GitHub Actions e GitHub Environments;
- Databricks Asset Bundles e Databricks Jobs;
- Apache Airflow (DAGs e provider Databricks) e Docker Compose;
- Unity Catalog e Delta Lake (COPY INTO, MERGE e Time Travel);
- Databricks SDK para Python (envio de arquivos a volumes).

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

Bibliotecas e serviços de terceiros integrados ao produto também são itens de configuração e devem ser versionados explicitamente (arquivo de dependências travado — lockfile).

- Análise automática de vulnerabilidades em dependências (SCA) integrada ao pipeline de CI.
- Atualizações de dependências propostas automaticamente (ex.: Dependabot/Renovate) e submetidas ao mesmo fluxo de PR.

Não há subcontratados. As dependências externas são tratadas assim:

- **Databricks Free Edition:** plataforma sem SLA, somente serverless, com um único SQL Warehouse, até 5 tarefas de job simultâneas, cota diária de uso compartilhada pelos dois ambientes e exclusão de contas inativas; por usar um único workspace, uma indisponibilidade afeta Desenvolvimento e Produção ao mesmo tempo. Como mitigação, cargas completas em Desenvolvimento são evitadas perto do horário de Produção e todo o projeto é reconstruível a partir do repositório (bundle, setup, docker-compose.yml e scripts da base fictícia). Como a plataforma bloqueia conexões de

saída, a extração é feita no sentido inverso: a máquina local envia os arquivos ao Databricks (decisão registrada em ADR).

- **Máquina local:** o PostgreSQL, o Airflow e o extrator dependem de um computador ligado, com Docker em execução, no horário da DAG (o Airflow e o PostgreSQL somam alguns GB de memória). Como mitigação, o extrator é incremental e recupera automaticamente os dias sem execução na próxima rodada, e o dashboard exibe a data da última atualização.
- **GitHub (plano gratuito):** proteção da main, Environments e aprovação manual de deploy só estão disponíveis em repositórios públicos; por isso o repositório é público, o que é viável por conter apenas dados fictícios e nenhum segredo.
- **Estrutura do Protheus:** as tabelas de origem são tratadas como contrato externo; alterações de colunas são detectadas na Bronze.
- **Dados e LGPD:** somente dados fictícios, com CPF/CNPJ gerados com dígito verificador inválido para não coincidir com pessoas ou empresas reais.

14. Quadro-Resumo do Pipeline CI/CD

Resumo do fluxo padrão, do commit à produção:

Etapa	Descrição	Gatilho
CI	Lint, testes, análise de segurança e validação do bundle.	Pull Request para a main
Merge	Squash merge na main.	CI verde
CD — Desenvolvimento	Deploy do bundle e execução do job com gates de qualidade.	Merge na main
Gate de Release	Tag vX.Y.Z e aprovação manual no Environment prd.	Sucesso em Desenvolvimento
CD — Produção	Deploy da mesma versão, smoke test e GitHub Release.	Aprovação do gate
Ambiente local	git checkout da tag e docker compose up --build (extrator, DAGs e Airflow).	Release publicada

Nota: *este plano adapta o template ao projeto PDC, mantendo os quatro pilares da GCS — identificação, controle de versão, controle de mudanças e auditoria/status — suportados por automação.*